

Mehrgo Driver — Order Complete Logic

End-to-end flow of how the driver app finalises a trip and posts the fare. Source: Android app, branch mehrgo_driver_app, version 1.9.1 (build 47).

1. Trigger

The driver is in an active trip (`OrderState.STARTED = 7`) and taps “**Finish trip**”. This calls `prepareOrderFinish()` in `MapFragment.kt:1364`.

2. Pre-finish refresh

Before showing the fare summary, the app refreshes the user from `GET user/me` (`MapFragment.kt:1366`) to pull:

- the latest active order (`user.orders[0]`)
- the client’s current bonus balance & bonus settings (so the deduction math is correct against server state)

Then `totalServicePrice` is summed from `order.services[].total` (`MapFragment.kt:1384`) and `calculateTotalPrice()` runs.

3. Fare computation (local taximeter)

Inside `setViewDialogTrackingFinish()` (`MapFragment.kt:1422`) — runs every time the summary dialog opens. The total fare is built in this order:

Component	Source / Formula
<code>startingPrice</code>	<code>order.startingPrice</code> — base fare / minimum
<code>additionalPrice</code>	<code>order.addPrice</code>
<code>totalServicePrice</code>	Sum of <code>order.services[].total</code>
<code>totalWaitingPrice</code>	Accrued from <code>MyTrackingService</code> while speed < 7 km/h, split into pre-arrival vs on-way at the tariff’s <code>priceOfWaiting</code> / <code>priceOfWaitingOnWay</code> rates
<code>totalTrackingPriceInCity</code>	km inside <code>branch.polygon</code> × <code>priceInCity</code> (or <code>tariff.distanceIntervals</code> ladder)
<code>totalTrackingPriceOutCity</code>	km outside polygon × <code>tariff.priceOfOut</code>

`totalPrice = startingPrice + additionalPrice + services + waiting + insideKm + outsideKm`
(`MapFragment.kt:1448`)

4. Multi-stop “shortcut” guard

For orders with **>= 2 destinations** (`MapFragment.kt:1447–1485`) the app picks between two prices:

```
distanceTracked          = distanceInCity + distanceOutCity    // what the driver actually drove
distanceBetweenLocations = order.distance * 1000               // server's planned route distance
distanceWay               = distance(lastFix -> order.locations.last())

if |distanceTracked - distanceBetweenLocations| > 1000 m OR distanceWay < 200 m:
    use the locally calculated taximeter total
else:
    use order.price (server-precomputed) + totalWaitingPrice
```

Plain English. If the driver clearly drove the planned route (within 1 km of the planned distance and ended within 200 m of the last drop-off), bill the agreed price. If they detoured or stopped short, bill the taximeter. This stops drivers from gaming the route, and also stops them being penalised when traffic forces a detour the taximeter would over-bill.

5. Promo code deduction

Source: MapFragment.kt:1494.

- `order.promoCode.usage.amount` can be either "10%" (percentage) or a flat number "5000".
- Percentage: `calculatedPromo = totalPrice / 100 × percent`
- Fixed: `calculatedPromo = min(amount, totalPrice)`

6. Bonus deduction

Source: MapFragment.kt:1524. Only applies if `order.useBonus == true`. Bonus cap (`maxAmount`) is either a percent or a flat number, applied to `(totalPrice - promo)`. Then:

```
calculatedBonusFinal = min(calculatedBonus, clientTotalBonus)
if clientTotalBonus < minAmount:
    calculatedBonusFinal = 0
```

So the bonus is bounded by **three** things: the tariff cap, what the client actually has, and the minimum threshold to use any bonus at all.

7. Final amount shown to driver

`finalTotalPrice = totalPrice - calculatedBonusFinal - calculatedPromo`

The driver sees the **gross total**, the bonus deduction, the promo deduction, and the **final amount to collect from the client**.

8. Submit

Source: MapFragment.kt:1568–1581.

```
POST order/complete?order_id={orderId}

Body (RequestOrderFinish):
{
  "distance":           "{km}",           // rounded km as string
  "latitude_finish":    {last GPS lat},    // MyTrackingService.lastLatLngWholeApp
  "longitude_finish":   {last GPS lon},
  "total_price":        "{totalPrice}",    // pre-deduction gross
  "waiting_time":       "{ms}",           // currentWaitTimeInMillis
  "execution_time":     "{ms}",           // currentTimeInMillis (total trip)
  "finish_address_id":  null,              // always null in current build
  "bonus_payment":      {calculatedBonusFinal}, // debit from client's bonus wallet
  "promo_code_payment": {calculatedPromo}   // debit from promo bucket
}
```

Server returns BaseResponse<User> — the refreshed user with new balance and cleared active orders.

9. Success side effects

Source: MapFragment.kt:1597–1612.

- **UserManager.saveUser(...)** — local cache updated with new balance.
- **commandStopTracking()** — sends ACTION_STOP_TRACKING intent to MyTrackingService: stops GPS, location-upload websocket, and clears taximeter counters.
- **stopRouteService()** — clears the active-trip route line and stops route polling.
- Both dialogs dismissed (dialogTrackingFinish, dialogFinishDestination).
- Map state reset: destinationLocation = null, placemarks + polyline cleared.
- findNavController().navigate(R.id.action_mapFragment_self) — reloads the map fragment so the UI is back to idle / online.

10. Error handling

Source: MapFragment.kt:1614–1639. Two error paths:

Server response	App behaviour
HTTP 402 (status: 402)	Show DialogPaymentError with the server message — typically “client cannot pay this amount with bonus / card”. Driver collects in cash. No retry.
Other 4xx with ErrorOrderFinishResponse payload	Server is rejecting the bonus calculation. The payload carries the correct clientTotalBonus, minAmount, maxAmount. App updates those values and re-renders the summary dialog so the driver can resubmit with the corrected bonus.

Why duplicate the bonus math client + server? The server is authoritative, but the client computes locally so the driver sees the price live while driving. On mismatch, the server quietly hands back the corrected settings and the client retries — the driver never sees a hard error.

Files involved

Layer	File
API	data/remote/ApiService.kt — line 212

Layer	File
Repository	data/repository/MainRepositoryImpl.kt — line 81
Use case	domain/use_case/main/OrderFinishUC.kt
Request DTO	domain/model/requests/RequestOrderFinish.kt
Error DTO	domain/model/error/ErrorOrderFinishResponse.kt
ViewModel	presentation/maps/yandex_map/MapViewModel.kt — line 65
UI / orchestration	presentation/maps/yandex_map/MapFragment.kt — lines 1364–1643

Document generated for internal review. Aligned with Android branch mehrgo_driver_app, commit 3e7291a6. Update this document if the RequestOrderFinish schema or the multi-stop guard thresholds change.